

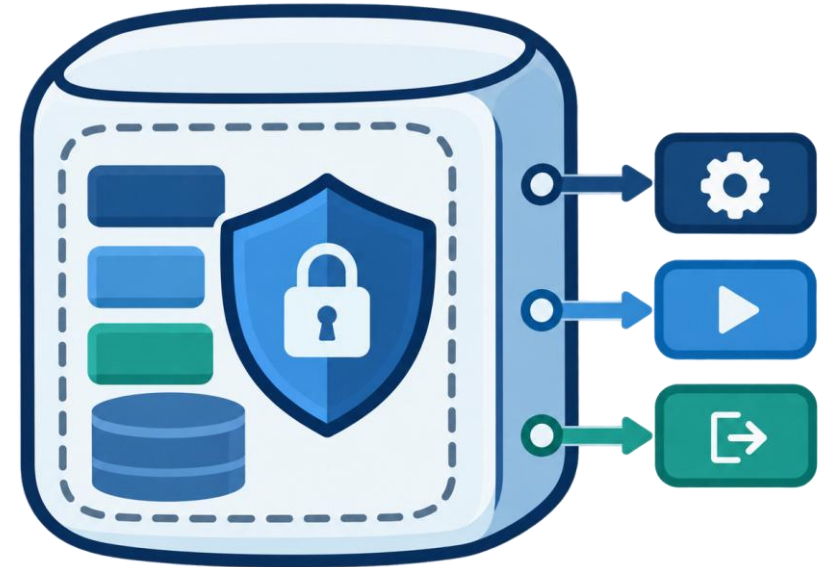
Access Modifiers & Encapsulation

Protecting object state and designing a clean public API

CSC241 - Object-Oriented Programming

Fall 2026 | COMSATS University Islamabad, Lahore Campus

Dr. Muhammad Shahid Bhatti
Assistant Professor • Department of Computer Science • Office H-17



PRIVATE STATE → CONTROLLED API
Clients use behavior; the object protects its rules.

Bridge from Lecture 4

Last lecture gave objects state and behavior. Today we decide who is allowed to touch that state.

Lecture 4

Fields store per-object state
Methods define behavior
Parameters & return types
Method overloading

Problem

A field can be changed directly from anywhere if we expose it. That means client code can bypass our rules.

Lecture 5

Control visibility
Hide representation
Expose purposeful operations
Keep objects valid

Key shift: a class is not just a bag of fields + methods; it is a boundary around state and behavior.

Lecture 5 scope from the CDF

Week 3 • Lecture 5 • Access Modifiers & Encapsulation • Quiz #1

- 1 public / private / protected / default access
- 2 Getters and setters
- 3 Why fields should be private
- 4 Information hiding and API design
- 5 Packages and controlling access across packages
- 6 Class invariants and validation rules for keeping objects in valid states

Boundary for today: we are not teaching constructors, static members, inheritance, or exception handling in depth yet.

Learning outcomes

By the end of the lecture, students should be able to...

1.

Choose an access modifier based on intended visibility

2.

Explain encapsulation vs. information hiding

3.

Refactor public fields into a controlled API

4.

Write getters/setters without blindly exposing everything

5.

Enforce simple class invariants through validation

6.

Reason about member access across Java packages

Teaching rhythm: predict → code → compile → run → explain → refactor.

Motivation: a class with public fields

This compiles — but does the object remain trustworthy?

STUDENT.JAVA

```
public class Student {  
    public String studentId;  
    public String name;  
    public double gpa;  
    public int completedCredits;  
}
```

DEMO.JAVA

```
Student s = new Student();  
s.studentId = "CIIT/SP26-BCS-005/LHR";  
s.name = "Abeer Amina";  
s.gpa = 3.72;  
s.completedCredits = 48;  
  
// Nothing stops this:  
s.gpa = -7.0;  
s.completedCredits = -100;
```

Ask the class: which lines are legal Java? Which lines are logically valid for a university student?

Live coding 1 — break the object on purpose

Let client code mutate fields directly, compile it, then inspect the state.

1

Create public fields

```
Student s = new Student();  
s.name = "Ali Ishtiaq";  
s.studentId = "CIIT/SP26-BCS-010/LHR";  
s.gpa = 3.40;  
s.completedCredits = 42;  
  
System.out.println(s.gpa);
```

2

Assign valid values

3

Assign nonsense

```
s.gpa = 8.9;  
s.completedCredits = -25;  
  
System.out.println(s.gpa);  
System.out.println(s.completedCredits);
```

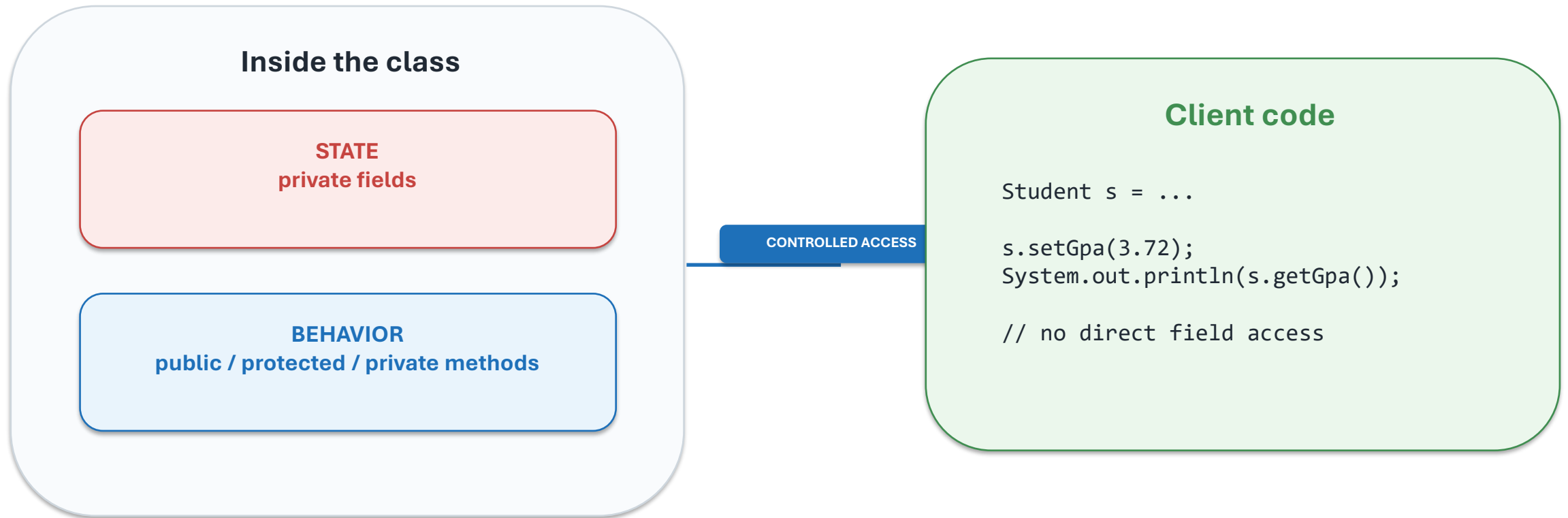
4

Print object state

Prediction before Run: compiler error, runtime error, or successful execution?

What is encapsulation?

One object packages state + behavior and controls how the outside world interacts with it.



Encapsulation is the boundary. Access modifiers are one mechanism for enforcing that boundary.

Encapsulation vs. information hiding

Related ideas — but not exactly the same.

Encapsulation

Bundle related data and operations into one abstraction — e.g., Student owns both GPA state and GPA-related behavior.

Information hiding

Conceal representation choices that clients do not need to know — e.g., whether GPA is stored as double, integer points, or calculated.

Good design goal: clients depend on what the object promises, not on how the object stores its data.

Think of the class as a contract: **“Here is what you may ask me to do.”** not **“Here is everything I store.”**

Four Java access levels at a glance

Visibility depends on class/package relationships.

Modifier	Same class	Same package	Subclass (other pkg)	Other package
public	✓	✓	✓	✓
protected	✓	✓	✓*	—
default	✓	✓	—	—
private	✓	—	—	—

* protected across packages is intended for subclasses. We will revisit the inheritance details later.

public — part of the externally visible API

Use public when other classes are expected to call or use the member.

STUDENT.JAVA

```
public class Student {  
    public String getName() {  
        return name;  
    }  
  
    public double getGpa() {  
        return gpa;  
    }  
}
```

Meaning

Accessible from client code wherever the Student type itself is accessible.

Use for

Operations that belong to the class contract: queries, commands, carefully chosen constants.

Do not infer

“Everything useful must be public.” A small API is usually easier to understand and maintain.

private — implementation detail inside the class

Private members are hidden from ordinary client code.

STUDENT.JAVA

```
public class Student {  
    private String studentId;  
    private String name;  
    private double gpa;  
  
    public double getGpa() {  
        return gpa;           // OK: same class  
    }  
}
```

DEMO.JAVA

```
Student s = new Student();  
System.out.println(s.gpa);  
//                ^^^  
// compile-time error:  
// gpa has private access in Student
```

Design interpretation: private says “this representation belongs to the class, not to its clients.”

default / package-private — visible within the package

When no modifier is written, Java grants package-level access.

STUDENT.JAVA

```
package edu.cuilahore.csc241.model;  
  
public class Student {  
    int semester;    // package-private  
    String section;  // package-private  
}
```

Same package

Other classes in edu.cuilahore.csc241.model can access semester and section.

Different package

A class in edu.cuilahore.csc241.app cannot access those members directly.

Package-private is useful for package-internal collaboration, but it is not the same as public.

protected — package access + inheritance-oriented access

Protected is broader than package-private, but narrower than public.

PERSON.JAVA

```
package edu.cuilahore.csc241.model;

public class Person {
    protected String name;
}
```

GRADUATESTUDENT.JAVA

```
package edu.cuilahore.csc241.grad;

public class GraduateStudent extends Person {
    void printName() {
        System.out.println(name); // OK
    }
}
```

Same package

protected behaves much like package-private for classes in the same package.

Other package

A subclass may access inherited protected state. Details become clearer when we study inheritance.

protected — preview only; details come with inheritance

For today, remember one safe rule: protected is not the same as public.

```
package p2;

public class GradStudent extends Person {
    void demo() {
        System.out.println(name); // OK

        Person p = new Person();
        // System.out.println(p.name); // NOT OK
    }
}
```

Rule of thumb for now

Outside the package, think of protected as access through the subclass relationship — not as public access.

Why mention this now?

It prevents the common misconception: “protected = public for subclasses everywhere.” The exact inheritance rules come later.

Access modifiers on classes vs. members

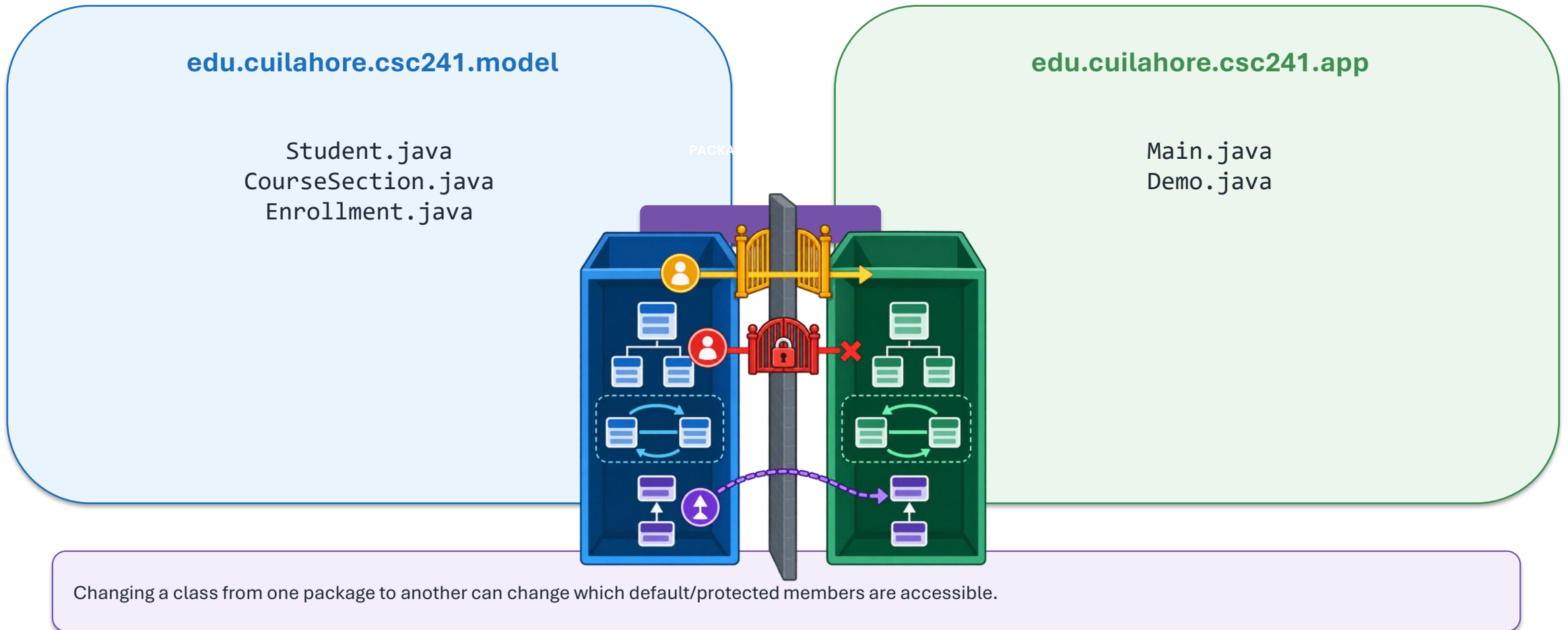
Top-level classes have fewer choices than fields/methods.

Declaration	Allowed?	Meaning
<code>public class Student</code>	✓	Accessible from other packages
<code>class Helper</code>	✓	Package-private top-level class
<code>private class Student</code>	✗	Not allowed for top-level class
<code>protected class Student</code>	✗	Not allowed for top-level class

For today, focus mainly on member access. Nested classes have additional possibilities and are not our topic yet.

Packages are part of the access-control story

A Java package is a namespace and a visibility boundary.



Live coding 2 — compile code across packages

Use the compiler to make visibility rules concrete.

PROJECT LAYOUT

```
src/  
└─ edu/cuilahore/csc241/  
    └─ model/  
        └─ Student.java  
    └─ app/  
        └─ Demo.java
```

TERMINAL

```
javac -d out \  
    src/edu/cuilahore/csc241/model/Student.java \  
    src/edu/cuilahore/csc241/app/Demo.java  
  
java -cp out edu.cuilahore.csc241.app.Demo
```

Classroom move

First make a field package-private and compile from app. Then make it public and recompile. Let the compiler reveal the rule.

Predict before compile: which access changes the error?

Cross-package access: let the compiler answer

model.Student is public, but its members may have different visibility.

STUDENT.JAVA

```
package edu.cuilahore.csc241.model;

public class Student {
    public String name;
    int semester;
    private double gpa;
}
```

DEMO.JAVA

```
package edu.cuilahore.csc241.app;

import edu.cuilahore.csc241.model.Student;

Student s = new Student();
s.name = "Abdul Rehman Azam";    // OK
// s.semester = 3;                // ERROR
// s.gpa = 3.5;                   // ERROR
```

Question: which restriction is caused by the package boundary, and which is caused by private?

Why fields should usually be private

A private field lets the class stay in control of its representation and rules.

Protect validity

Reject impossible values before state changes

Reduce coupling

Clients depend on methods, not storage details

Change implementation

Rename/recompute fields without rewriting every client

Centralize rules

Validation, logging, derived behavior live in one place

Private-by-default is a useful design habit — then open only what the class contract genuinely needs.

Getter: controlled read access

A getter exposes information without exposing the field itself.

STUDENT.JAVA

```
public class Student {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
}
```

Client view

```
String n = s.getName();
```

The caller receives the value through a method call.

Design benefit

The class can later compute, format, or derive a result without changing client syntax.

A getter is part of the API. Do not generate getters for every field automatically without asking whether clients need them.

Setter: controlled write access

A setter can validate before allowing state to change.

STUDENT.JAVA

```
public boolean setGpa(double newGpa) {  
    if (newGpa < 0.0 || newGpa > 4.0) {  
        return false;  
    }  
    gpa = newGpa;  
    return true;  
}
```

Why this is better

Every caller goes through one rule. Invalid assignments cannot silently corrupt the object.

For this lecture

Use true/false to report whether the state change succeeded. Exception handling comes later.

Not every private field needs both get and set

Encapsulation is not “make fields private, then expose them all again.”

Read-only from clients

```
private String studentId;  
public String getStudentId()
```

No setter if ID should not change.

Controlled command

```
private int completedCredits;  
public void addCredits(int c)
```

Better than arbitrary `setCredits(...)`.

Derived property

```
public boolean isEligibleForFyp()
```

Could compute from credits; no field is required.

API question: what meaningful operation should the client perform? That is usually better than exposing storage mechanics.

Live coding 3 — refactor public fields into an API

Start from the broken Student class and improve it step by step.

STUDENT.JAVA

```
public class Student {  
    private String studentId;  
    private String name;  
    private double gpa;  
  
    public String getStudentId() { return studentId; }  
    public String getName()      { return name; }  
    public double getGpa()       { return gpa; }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public void setGpa(double gpa) {  
        /* add validation next */  
        this.gpa = gpa;  
    }  
}
```

Class participation

Ask students which field should have no setter and why.

Compile checkpoint

Try old client code `s.gpa = 3.7;` and observe the compiler error.

Refactor checkpoint

Replace direct field access with `getGpa()/setGpa(...)`.

Validation belongs where state changes

One rule, one place, every caller.

```
public boolean setGpa(double gpa) {  
    if (gpa < 0.0 || gpa > 4.0) {  
        return false;  
    }  
    this.gpa = gpa;  
    return true;  
}
```

Without validation

Any caller can push the object into an impossible state.

With validation

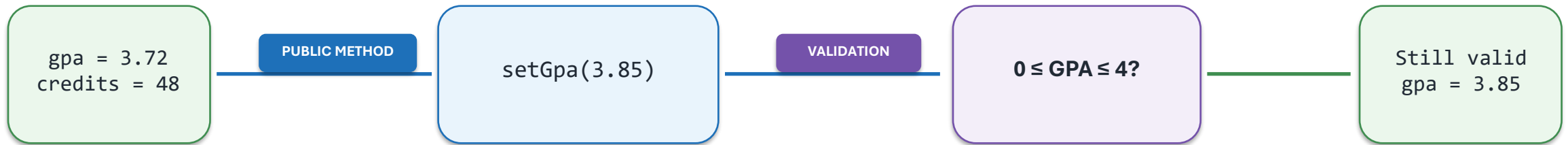
Every path to GPA modification obeys the same 0.0–4.0 rule.

Design principle: the object should defend its own rules rather than trust every caller to remember them.

Class invariant: a condition that should always remain true

After construction and after every public operation, the object should still be valid.

Valid object



Invariant = a promise about the object's state, not just a check performed once.

Possible invariants for our Student class

Rules should come from the domain, not from syntax.

studentId

not null / not blank

name

not null / not blank

gpa

$0.0 \leq \text{gpa} \leq 4.0$

completedCredits

$\text{completedCredits} \geq 0$

Ask students: which of these rules could change if university policy changes? Which are intrinsic to the data model?

Public fields let clients bypass invariants

The class cannot defend rules it does not control.

```
public class Student {  
    public double gpa;  
}  
  
Student a = new Student();  
a.gpa = 3.7;    // valid  
a.gpa = 9.4;    // Java permits it  
a.gpa = -2;     // Java permits it
```

Problem

The Student class has no single place to enforce “GPA must be between 0 and 4.”

Result

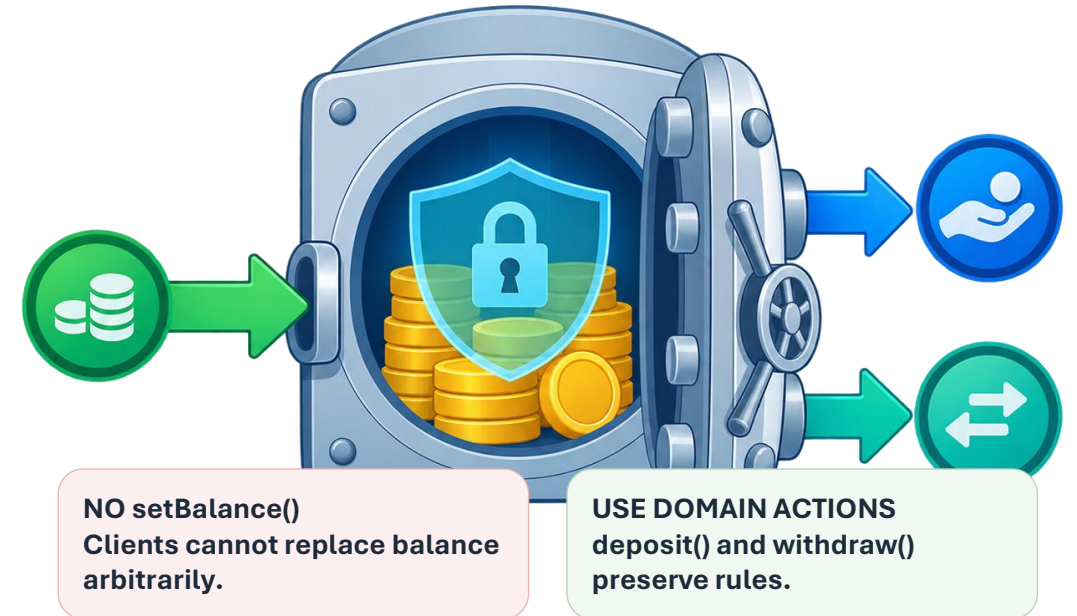
Correctness depends on every piece of client code behaving perfectly forever.

Encapsulation changes correctness from a caller convention into a class responsibility.

BankAccount — behavior beats raw setters

Let the object expose meaningful operations instead of arbitrary state replacement.

```
public class BankAccount {  
    private double balance;  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public boolean deposit(double amount) {  
        if (amount <= 0) return false;  
        balance += amount;  
        return true;  
    }  
  
    public boolean withdraw(double amount) {  
        if (amount <= 0 || amount > balance)  
            return false;  
        balance -= amount;  
        return true;  
    }  
}
```



Class question: Why is getBalance() useful while setBalance(...) is a poor API choice?

API design: public surface vs. private implementation

Clients should see a small, meaningful surface.

Student

IMPLEMENTATION

```
private String studentId;  
private String name;  
private double gpa;  
private int completedCredits;
```

PUBLIC API

```
getName()  
getGpa()  
setGpa(double)  
addCredits(int)  
isEligibleForFyp()
```

Keep implementation freedom on the left; make promises deliberately on the right.

Encapsulation reduces coupling

Client code depends on stable methods instead of internal representation.

BEFORE

```
// Client code
System.out.println(s.getCompletedCredits());
```

CLIENT UNCHANGED

The getter acts as a seam: storage can change while the external contract stays stable.

```
// Internal implementation v1
private int completedCredits;

public int getCompletedCredits() {
    return completedCredits;
}
```

```
// Internal implementation v2
private int passedCreditHours;

public int getCompletedCredits() {
    return passedCreditHours;
}
```

Live coding 4 — transfer between two BankAccount objects

Combine private state, references, this, validation, and object-to-object interaction.

```
public boolean transferTo(BankAccount target,  
                          double amount) {  
  
    if (target == null || target == this)  
        return false;  
  
    if (!withdraw(amount))  
        return false;  
  
    target.balance += amount;  
    return true;  
}
```

1 Predict
What happens when target is null?

2 Explain private
Why can BankAccount code access target.balance?

3 Compare designs
Would target.deposit(amount) be clearer?

Key idea
private is class-level access, not “this object only.”

Common misconceptions

Small syntax mistakes often reveal a deeper design misunderstanding.

“private means no one can use it”

Clients can use behavior that internally reads/writes private fields.

“getter/setter = encapsulation”

Only if the API actually controls and communicates access.

“default means public”

No modifier means package-private, not public.

“protected is public to subclasses”

Across packages, access is tied to the subclass relationship.

Rule of thumb: expose the smallest meaningful API that lets the object preserve its own rules.

Compiler clinic — predict each line

Assume Demo is in a different package and is not a subclass.

```
public class Student {  
    public String name;  
    protected int semester;  
    double gpa;  
    private String studentId;  
}
```

s.name = "Hania";

COMPILES

s.semester = 3;

ERROR

s.gpa = 3.6;

ERROR

s.studentId = "...";

ERROR

Then move Demo into the same package. Which answers change?

Quiz #1 — access and encapsulation check

Use as an in-class quiz, quick poll, or discussion before submission.

1

1. Which modifier gives access only inside the class implementation?

2

2. What does “no modifier” mean for a field?

3

3. Why is `public double gpa;` weaker than `private double gpa + setGpa(...)`?

4

4. Does a private field require a setter? Explain.

5

5. A class in another package is not a subclass. Can it access a protected member?

6

6. State one invariant for Student and identify where you would enforce it.

Do not reveal answers immediately — ask students to justify each choice using the access table and object boundary.

Integrated design challenge — CourseSection

Design an encapsulated class before writing the full code.

Requirements

- courseCode cannot be blank
- capacity > 0
- registered cannot be negative
- registered cannot exceed capacity
- clients should enroll through enroll()

Your design tasks

1. Choose field modifiers
2. Choose getters
3. Decide whether setters are needed
4. Write enroll()
5. State the invariants

CLIENT VIEW

```
CourseSection oop = new CourseSection();

// Desired client interaction:
System.out.println(oop.getAvailableSeats());
oop.enroll();

// Avoid:
// oop.registered++;
```

Pair task: design the public surface first; then decide what must remain private.

Lecture 5 recap & exit ticket

Encapsulation turns object correctness into a class responsibility.

Access

public / protected / default / private

Boundary

hide representation; expose purposeful API

State

fields usually private

API

getters/setters only when justified

Validity

validate at state-change boundaries

Packages

visibility changes across package boundaries

Exit ticket

1. In one sentence, distinguish encapsulation from information hiding.
2. Name one field in Student that should not have a setter, and justify.
3. What compiler error would you intentionally create to demonstrate package-private access?

Next lecture: Constructors — create objects in a valid state from the moment they are created.